

Seat Ase? System Design, Final Version

A Dhaka Tesla Pool MVP. Find a seat. Find a ride. Stay strangers.

Document	System design, version 7 (final). Describes the system as released.
Release described	v1.0.0, commit 125ab61, on the master, pre-release and release/v1.0.0 branches
Date	30 September 2026
Author	Shajid Shahriar
Purpose	RoBenDevs SWE Intern take-home: the Dhaka Tesla Pool MVP
Live application	https://seat-ase-web.vercel.app
Stack	Next.js, Express, PostgreSQL, Docker. JavaScript throughout, Zod for validation.

Summary

Seat Ase? is built, deployed and released as v1.0.0. It lets passengers in Dhaka request a shared or private ride in a "Tesla" (a three-wheeled battery rickshaw), lets drivers choose whom to take, and pools compatible passengers into one vehicle with a separate fare for each. The name is Bangla for "is there a seat?".

The central design decision is that every rule that could hurt someone if broken is enforced on the server and in the database, and the screens only display the result. Seat capacity, status changes, fares, ownership of rides and the women-only rule are checked inside Postgres transactions. A Postgres CHECK constraint stops a three-seat Tesla from carrying four people even if the application code has a bug, and a live Seat Race page in the app shows two simultaneous requests competing for the last seat.

The design went through seven versions. Version 6 was written before any code and then reviewed twice for loopholes. This version 7 is written after the build, so it records what exists, why each decision was made, and where the build changed the design. After the build, a read-only review of the finished code produced 66 findings. All ten highest-severity findings and most of the high-severity ones were fixed and tested before release. The remainder are documented as known limitations.

At release the system had 281 automated tests in 27 files, all run against a real PostgreSQL database, a Postman collection of 52 requests with 61 assertions, and a public deployment on free tiers (Vercel, Render and Neon). Sections near the end of this document give the build roadmap, the history of design changes, the review results and the known limitations.

Background and problem

The brief asks for a small version of Uber Pool for Dhaka. At 8:41 AM at Banani Road 11, a driver named Jashim waits beside Bullet, his three-seat Tesla. Nusrat books a ride to Mohakhali. Two minutes later Rafiq, a stranger, books almost the same route to Gulshan 1. The app must decide whether they can share the vehicle, split the fare fairly, and keep each person's fare and status private from the other. Shirin then tries to take the last seat, and the app must never let the vehicle exceed its capacity.

In Dhaka a shared Tesla does not come to the door. Passengers walk to a stand, the driver calls

out destinations, and he waits until his seats fill. The design copies that behaviour and removes the guesswork: before a passenger walks to a stand, the app shows whether a Tesla is there, whether it has room, and what the fare will be.

The cast

The brief asks for the story characters as real test data. They are used identically in the seed data, the tests, the README and the demo, so the claims in the documents match what runs.

Name	Role	Details, fixed everywhere
Jashim	Driver	Drives Bullet, 3 seats, plate DHAKA-METRO-GA-11-1111
Mokbul	Driver, added for this design	Drives Toofan, 3 seats, plate DHAKA-METRO-GA-22-2222. Needed for the two-driver race and the women-only ride.
Nusrat	Passenger, female	Banani to Mohakhali, 1 seat
Rafiq	Passenger, male	Banani to Gulshan 1, 1 seat (2 seats in the last-seat race)
Shirin	Passenger, female	Banani to Mohakhali, 1 seat, usually asks for women-only

Every account uses the password password123 in the demo, and all five are pre-verified for phone and NID.

What is graded

The brief grades product understanding, process (git history and following instructions), backend and database design, frontend flows, Docker and deployment, testing and documentation, and the author's ability to explain and change the code. It states that a small clean MVP that follows the process can outscore a large one that does not. That shaped the design goals below.

Design goals

- Enforce seats, statuses, fares and permissions in the database and API, so a screen or a buggy client cannot break them.
- Give every choice a written reason, an alternative that was considered and a condition under which it would be revisited.
- Prove the hardest requirement, the last-seat race, with a test and a live demonstration.
- Keep the scope small and state its limits.
- Keep one source of truth for each rule: the status transition table, the shared Zod schemas and the cast file.

Scope

In scope

Passenger and driver signup and login with simulated phone and NID verification. Driver vehicle

registration and going online or offline with a declared area. Ride requests, shared or private, with an optional women-only setting for female passengers. Driver-side pooling with fit and does-not-fit reasons. The full trip lifecycle: arrival, boarding, start, per-passenger drop-off, no-show and cancellation. Per-passenger fares in integer poysha with a stored breakdown. Live updates. History and receipts. A demo Seat Race. Docker, tests, deployment and documentation.

Out of scope

Real road routing and driver GPS tracking, push notifications, chat between driver and passenger, ratings, an admin panel, a payment gateway or wallet (payment is cash), real SMS delivery and a real ID registry, a female-driver preference and an SOS button, picking up passengers at several spots, refilling a seat mid-trip, and cancelling a started ride from the driver's side. The last three need live location and routing, which the brief tells candidates not to build. Redis, queues and microservices are also left out on purpose, and the scaling section explains when each would be added.

Assumptions

The brief leaves some details open and asks that assumptions be documented and applied consistently. These are the assumptions in the final design. Those marked changed differ from version 6.

No.	Assumption	Reason
A1	A Tesla is a three-wheeled battery rickshaw. Drivers register their own vehicle without document checks.	Matches the story. Verification is outside the MVP.
A2	Drivers choose which requests to accept. The app labels each request as fitting or not fitting and gives the reason.	Keeps the driver in control while the server still enforces the rules.
A3	No kick-out. Before accepting, a driver may skip anyone. After accepting, he can only cancel the whole ride before it starts, or mark a no-show after waiting five minutes. Changed: he also cannot re-accept a booking he cancelled.	Stops a driver from removing a person for their looks or gender after that person stopped looking for another ride.
A4	Dhaka is modelled as 12 zones with a stored distance table.	The brief says not to build real routing. Fares stay checkable by hand.
A5	Pool rule: same pickup stand, drop-offs within 3 km of each other, enough seats, ride not yet arrived, plus the women-only and private rules.	Limits detours to roughly 10 to 15 minutes in Dhaka traffic.

No.	Assumption	Reason
A6	Women-only is chosen by the passenger. It admits only rides where everyone is female and blocks men from joining afterwards. Gender is self-declared, set at signup and never editable.	Safety stays with the person it protects. The weakness is documented.
A7	Private hire takes the whole Tesla. Changed: the private fare is the seat fare multiplied by 3, whatever the vehicle's real capacity, and it never rises after the quote.	Private is never cheaper than sharing, and a quoted price never goes up.
A8	Phone verification is simulated. The code is shown on screen in demo mode.	The brief forbids paid services.
A9	The group locks when the driver arrives. Fares lock when the trip starts. Getting off early does not change the fare.	Predictable and fair.
A10	A passenger can cancel before the driver arrives, and after arrival only if the driver has not started within 15 minutes. A driver cannot cancel a started ride.	Protects the driver's time and frees passengers from a vanished driver.
A11	Each passenger is dropped at their own stop. The ride completes after the last drop.	How pooling works in practice.
A12	The database allows 1 to 6 seats per booking and the screen offers 1 to 4. One active booking per passenger. One Tesla per driver.	Prevents spam and confusion. Changed: version 6 said a maximum of 3.
A13	A request that no driver accepts expires after 15 minutes.	Nobody waits forever.
A14	Payment is cash. Money is stored as integer poysha.	Allowed by the brief. Exact arithmetic.
A15	The last-seat race happens through the driver's accept.	Consistent with A2. The database protection is the same either way.
A16	Changed: every ride, shared or private, is picked up at the nearest Tesla stand. Version 6 gave private rides a door pickup.	One meeting point for everyone, and no home location is ever stored.

No.	Assumption	Reason
A17	The driver's area is self-declared, not GPS-tracked. After the last drop-off it becomes the zone where he ended up.	No GPS or routing in scope.
A18	No pickups at several spots and no mid-trip seat refills.	Both need live driver location and routing.
A19	New: if a driver accepts but does not press arrived within 15 minutes, the ride is cancelled and the passengers return to the waiting list.	Closes a stuck state the design missed.
A20	New: pickup and drop-off are chosen by searching a place. Free map pins were removed.	A tapped point has no name to show and no geocoder is available.
A21	New: sessions are stateless seven-day sliding tokens and an account may be signed in on any number of devices.	Simple and correct. A log-out-everywhere feature is listed as future work.

Product decisions

Question	Decision	Reason
Who decides who rides?	The driver. He sees waiting requests, each labelled as fitting or not with a reason. The server re-checks when he taps accept.	No matching engine to build, and the rules are still enforced.
How do people choose places?	By searching a seeded list of about 50 Dhaka places. Every place belongs to a zone.	Feels like a real app, costs nothing, and keeps fares checkable by hand.
Where are rides picked up?	At the nearest real Tesla stand, one to three per zone, with the walk distance shown first.	This is how shared Teslas already work in Dhaka.
How does the app know where Teslas are?	It does not track GPS. Drivers declare an area. Passengers see a count such as "1 Tesla online near Banani".	A simulation that is stated as one. Real tracking is part of the scaling design.
How do screens update?	Server-Sent Events, with polling as a fallback.	Updates flow only from server to browser.
Driver signup	Account, then login, then add a vehicle with a name, a unique registration number and 1 to 6 seats. A driver cannot go online without a vehicle and, as built, must verify their phone first.	No document upload or admin approval in the MVP.

Question	Decision	Reason
Phone verification	Simulated one-time code. The table, hashing, five-minute expiry and five-attempt limit are real. Only delivery is faked.	Real SMS costs money.
Identity	Simulated NID check at signup: 10, 13 or 17 digits. Only a keyed hash and the last four digits are stored. Gender is set at signup and locked.	Supports the women-only feature. One NID allows one account.
Eight people request and Bullet has three seats	Nobody is auto-booked. All eight wait. Jashim and Mokbul accept from the same list and the rest keep waiting.	Nobody is in the car until accepted, so there is nothing to remove.

System architecture

The system has four parts. The diagram below shows the request path and the return path for live updates. The brief warns against adding Redis, Kafka, Kubernetes or microservices to look advanced, and none of them is used.

Browser →	Next.js (Vercel) →	Express API (Render) →	PostgreSQL (Neon)
Nusrat's phone, Jashim's phone	Pages, forms, map. Forwards every /api/* call.	Login, validation, pooling rules, fares, status changes, permissions	All data. Enforces seat limits, uniqueness and valid links.
Return path: after a change is saved, Express sends a small "ride changed" nudge over Server-Sent Events to the browsers concerned. The nudge holds no data. The browser then re-fetches through the normal API.			

What each part does and must not do

Part	Job	Must not
Browser	Shows screens and sends clicks.	Be trusted. Anything from a browser can be faked.
Next.js	Pages, forms, the map, and loading, empty and error states. Forwards /api/* calls to Express.	Hold business rules. It only displays.
Express API	Login, validation, ride rules, seat counting, fares, status changes, permissions and live nudges.	Trust the frontend's arithmetic. It recalculates every fare.
PostgreSQL	Stores everything and acts as the last line of defence with constraints, unique indexes and foreign keys.	Nothing. It is the source of truth.

Why Next.js forwards the calls

If the browser called a separate API site directly, the login cookie would be a cross-site cookie, which browsers treat with suspicion. Next.js therefore rewrites every `/api/*` request to the Express URL on the server side. To the browser there is one site, so the `httpOnly` cookie works without special settings, and no CORS headers are needed at all. The rewrite target is read when the web app is built, so the address is passed as a build argument in Docker and as a build environment variable on Vercel. All user data is fetched on the client with TanStack Query, because server-rendered fetches do not carry the login cookie.

Inside the API

Order	Stage	Job
1	Route	Maps a URL to a controller and attaches the middleware. No logic.
2	Request ID and logging	Gives every request an ID that appears on every log line. Request bodies are never logged.
3	Security headers and rate limits	Helmet, and rate limiting on login, signup and code sending.
4	Authentication	Verifies the JWT cookie and renews it (sliding session).
5	Role and verification check	<code>requireRole</code> for driver or passenger endpoints, <code>requireVerified</code> for actions that need a verified phone. Verification status is read from the database, never from the token.
6	Validation	A shared Zod schema checks the body, query and URL parameters. A malformed id returns 400.
7	Controller	Thin. Reads the request, calls a service and sends the response. No SQL.
8	Service	All business rules: matching, fares, status transitions, cancellations. Fare and matching are pure functions, so they are easy to test.
9	Transaction	Row lock, conditional updates and the CHECK constraint, inside one database transaction.

Order	Stage	Job
10	Notify	After the transaction commits, sends live nudges to the users affected.

Code layout

Folder	Contents
api/src/routes, controllers	URL wiring and thin request handlers.
api/src/services	Business rules: authentication, OTP, vehicles, places, fares, matching, ride requests, driver ride actions, the status transition table, scenarios for the demo.
api/src/middleware	Authentication, role and verification guards, validation, error handling, request ID, demo key guard.
api/src/db	Drizzle schema, migrations (drizzle/), and seed data for Dhaka, the drivers and the passengers.
api/src/realtime	The connection registry, the notify function and the table of who is nudged for each event.
api/src/config, lib, test	Environment validation, small helpers (hashing, geography, logging) and test setup.
web/app, components, lib	Next.js pages, UI components, the API client, query hooks and the live-update hook.
shared	Zod schemas and constants used by both the API and the web app: one definition of valid data.
postman	A runnable collection covering the whole demo flow.

Technology choices

For every tool the brief did not mandate, the table gives what was chosen, what else was considered, why the choice fits a ride-pooling MVP, and what would justify switching.

Decision	Chosen	Alternatives	Why it fits, and when to switch
Language	JavaScript with Zod. No TypeScript.	TypeScript	The dangerous bugs are overbooking, illegal status jumps and touching another person's ride. They happen with real data at run time. TypeScript checks code while it is written and disappears at run time. Zod checks every real request and the database checks

Decision	Chosen	Alternatives	Why it fits, and when to switch
			every real write. Switch for a larger team or a long-lived codebase.
Frontend	Next.js 16 (App Router), React 19, Tailwind CSS 4	React with Vite	File-based routing, a built-in rewrite for same-origin API calls, and first-class free hosting on Vercel. Vite would be lighter if no rewrite were needed.
Backend	Express 5 on Node 22	NestJS, Fastify, Next.js route handlers	Small enough that every line can be explained. A separate app keeps a clean split between screens and rules, and SSE is easy in Express. Node 22 replaced the Node 20 in version 6 because Node 20 reached end of life in April 2026. Switch to NestJS for enforced structure in a larger team, or to Fastify at high traffic.
Database	PostgreSQL 16	MongoDB, MySQL, SQLite	The problem is relationships and safe counting. Postgres provides foreign keys, CHECK constraints, partial unique indexes, transactions and row locks. In MongoDB the seat rule would live in application code and each code path would have to remember it. Switch or add PostGIS for heavy geographic search.
ORM and migrations	Drizzle ORM 0.45 with drizzle-kit 0.31	Prisma, Knex, raw pg	Queries read close to SQL. CHECK constraints, enums and partial indexes are written in the schema

Decision	Chosen	Alternatives	Why it fits, and when to switch
			file. drizzle-kit generates plain .sql migrations that a reviewer can read, and Drizzle supports SELECT ... FOR UPDATE. Fall back to Knex if Drizzle proves unfamiliar.
Authentication	Own bcrypt password hashing and a JWT in an httpOnly cookie, with the role in the token	Auth.js, Clerk, Supabase Auth, database sessions	The rules live in Express, so a library built for Next.js would split login across two apps. The implementation is short. Switch for social login, password-reset email, two-factor or log-out-everywhere.
Phone verification	Simulated one-time code behind one function, sendOtp()	An SMS gateway, Firebase phone auth, no verification	The brief forbids paying. Skipping verification would lose a real safety step. Only delivery is faked. Plug a Bangladeshi SMS provider into sendOtp() at launch.
Validation	Zod 4 in a package shared by web and API	Joi, express-validator	One schema serves the form and the API. Nothing realistic would trigger a switch.
Live updates	Server-Sent Events with a polling fallback	WebSockets, plain polling, Pusher	Updates flow only from server to browser. SSE is built into browsers, reconnects by itself and needs no library. Use WebSockets for driver and passenger chat, and a shared channel for many servers.
Map	MapLibre GL JS with OpenFreeMap vector tiles. Changed from Leaflet with OpenStreetMap tiles.	Google Maps, Mapbox, Leaflet	Free, no key and no rate limit on the map. The style can be recoloured to match the app, and one persistent

Decision	Chosen	Alternatives	Why it fits, and when to switch
			map sits behind every screen. A real launch would use a geocoding API with a budget and real road distances.
Frontend data	TanStack Query 5	fetch with useEffect, SWR	Provides loading, error and empty states, caching, and a single call to mark data stale, which is what an SSE nudge needs.
API style	REST with JSON and action endpoints such as POST /driver/ride/start	GraphQL, one generic PATCH status endpoint	The data is simple and the screens are fixed. Each action has its own permission check and its own allowed-from status.
Tests	Vitest 4 and Supertest against a real Postgres	Jest, a mocked database	The riskiest code is database behaviour, and a fake database cannot prove that two requests will not overbook. Add Playwright if the frontend grows.
Hosting	Vercel for the web app, a Render free web service built from the API Dockerfile, Neon free Postgres	Everything on Render, Railway, Fly.io, a VPS	Each is free without a card at this size. Render's free Postgres is deleted after 30 days, so Neon holds the data. The API runs from the same Dockerfile locally and online. Move to a paid tier when real users arrive.
Logging	winston with JSON output and a request ID on every line	console.log, pino	Familiar and easy to defend. At this traffic pino's speed advantage does not matter. Send logs to a central tool once there are many servers.
Money	Integer poysha in integer columns	NUMERIC, floating point	In JavaScript 0.1 + 0.2 is

Decision	Chosen	Alternatives	Why it fits, and when to switch
			0.30000000000000004. Integers are exact in JavaScript and Postgres. Use NUMERIC with a money library for several currencies or fractional poysha.
Containers	Docker Compose with three services: db, api and web	A single container, Kubernetes	One command starts everything. Every published port can be changed with an environment variable.

Geography, stands and zones

The brief tells candidates not to fight map APIs. The design uses three layers of location, all backed by data the application owns.

Layer	What it is	Used for
Exact point	Latitude and longitude of a searched place.	Finding the nearest stand.
Tesla stand	A real pickup spot. The seed has 23 stands, one to three per zone, and 19 landmarks, 42 places in all.	Where every ride is picked up. Pooling rule R1.
Zone	One of 12 areas: Banani, Gulshan 1, Gulshan 2, Mohakhali, Tejgaon, Farmgate, Dhanmondi, Mirpur, Uttara, Bashundhara, Badda and Motijheel.	Fare distance, the drop-off rule R2 and the driver's area.

Search and snapping

Place search (GET /places?q=) is answered from the database. Wildcard characters in the query are escaped, so a search for "%" matches nothing. The server can snap any coordinate to the nearest stand using the haversine distance, and reports the distance and the walk time. Walk time is the straight-line distance times 1.3 divided by 5 km/h. Coordinates more than 3 km from the nearest zone centre are rejected as outside the service area, so a pin in Chattogram or at latitude 0, longitude 0 cannot produce a trip. The screens no longer offer free map pins, so the snapping is used only for searched places, but the API still accepts any coordinates and the tests use that.

Distances between zones

A stored zone distance table (approximate road kilometres, rounded to 0.5) is used for fares and

for the drop-off rule. With 12 zones there are 66 pairs. Six pairs that the demo scenarios depend on are fixed by hand so the evaluator can check fares in seconds. The other 60 pairs were estimated from straight-line distance multiplied by 1.3 and rounded, and then passed through a shortest-path pass so that no detour through a third zone is ever shorter than the direct distance. The table is therefore an estimate, and no pair beyond the six above was measured. A test checks that every pair exists.

From	To	Distance
Banani	Mohakhali	3.0 km
Banani	Gulshan 1	2.0 km
Mohakhali	Gulshan 1	2.5 km
Banani	Tejgaon	4.0 km
Mohakhali	Tejgaon	2.0 km
Gulshan 1	Tejgaon	3.0 km

A pre-release check found that the seeded centre of Gulshan 2 sat about 160 m from Banani's centre, which made the Banani to Gulshan 2 distance 0 km and sent two places to the wrong zone. The centre was corrected, the distances were regenerated, and the seed now updates existing rows, so already-deployed databases receive the fix.

Where the drivers are

The system does not track driver location. A driver declares an area when he goes online, and after his last drop-off the area becomes the zone where he ended up. The passenger sees a count of Teslas she could get into. A car is counted when it is online, has been seen in the last five minutes, is in her area, and either has no active ride or has an open shared ride at her stand with a seat still free. Cars with an arrived or started ride, a full ride or a private ride are not counted. The count appears as soon as a pickup is chosen, before a drop-off is picked, and as a bubble over the stand on the map. In production the driver's phone would send its location every few seconds to a location service, described in the scaling section.

Pooling rules

The driver chooses whom to take, but the app labels every waiting request as fitting or not fitting his current ride and shows every failing reason. A request fits when all six rules hold. The check is a pure function, and the server repeats it inside the accept transaction on fresh data.

Rule	Statement	Reason
R1	The request has the same pickup stand as the ride.	One meeting point, as at a real stand.
R2	Its drop-off zone is within 3 km of every drop-off already in the ride. The same zone counts as 0 km. The limit is the constant MAX_DROP_KM.	Nobody's trip gets a large detour.
R3	The seats requested do not exceed the free seats.	Capacity.
R4	The ride has not reached arrived. Its status is still OPEN.	Once the driver is at the stand the group is locked.

Rule	Statement	Reason
R5	Women-only. A women-only request can join a ride only if every passenger already in it is female. A ride that contains any women-only booking accepts only female passengers afterwards.	Safety, controlled by passengers.
R6	Private. A private request can be accepted only into an empty Tesla, and a private ride accepts nobody else.	Hiring the whole Tesla means the whole Tesla.

Women-only

Only passengers turn women-only on. The driver has no switch, and the system never marks a ride women-only because two women happen to share it. A ride counts as women-only while it contains at least one women-only booking. This is worked out from the bookings inside the same locked transaction as the accept, so it is always current, and if the women-only passenger cancels or is a no-show the restriction disappears. Only accounts with gender FEMALE see the toggle, and the server rejects the flag from anyone else. Gender is self-declared, which is the main weakness, and the limitations section names the production fix.

Ride currently has	New request	Result
Nobody	Shirin, women-only	Fits. From then on only women can join.
Nusrat and another woman, neither women-only	Shirin, women-only	Fits, since everyone on board is female. Now no men can join.
Nusrat and Rafiq	Shirin, women-only	Does not fit. The ride has a male passenger.
Shirin, women-only	Rafiq	Does not fit. Women-only ride.
Shirin, women-only	Nusrat, not women-only	Fits, since she is female.
Anyone	A private request	Rule R6 decides. Private needs an empty Tesla.

The five scenarios

The story is covered by five fixed scenarios. They are scripts, not seed data, because the database rules allow only one active ride per vehicle and one active booking per passenger, so they cannot all exist at once. The tests build each one from the same cast file. The demo endpoint offers reset, the last-seat race and the two-drivers race.

Scenario	What happens	What it proves
A. Morning pool	Jashim accepts Nusrat, then Rafiq, who fits. Shirin's women-only request does not fit because Rafiq is on board. A request to Dhanmondi does not fit because it is far outside 3 km.	The pooling rules, and fares of 96.25 and 77.50.

Scenario	What happens	What it proves
B. Last-seat race	Bullet carries Rafiq and a colleague (2 seats). Nusrat and Shirin, one seat each, race for the last seat.	Concurrency.
C. Women-only	Mokbul accepts Shirin's women-only request into Toofan. Male requests then show as a women-only ride.	Rule R5.
D. Private	Nusrat hires the whole of Bullet privately for Tk 345.00. Nobody else can join.	Private hire and rule R6.
E. Three stops	Nusrat (Mohakhali), Rafiq (Gulshan 1) and Shirin (Tejgaon, not women-only this trip) share Bullet from the Banani Road 11 stand. Jashim drops them in the suggested order and collects Tk 288.75.	Drop-off order, per-passenger drops and fares.

In scenario E the suggested drop order is nearest first from the pickup zone: Rafiq at 2.0 km, Nusrat at 3.0 km, then Shirin at 4.0 km. It is only a suggestion, because it comes from the zone distance table and no real route is known, and the driver knows Dhaka's roads better. The ride completes when the last passenger is dropped, and the driver's area becomes Tejgaon.

What the driver sees

The driver sees waiting requests for the stands in his declared area, grouped by stand, oldest first, with how long each person has waited. After his first accept the list is filtered to the ride's stand. Each card shows the stand, the seats, whether it is shared, private or women-only, and the fares he would collect, as pooled and alone with the distance. A private request reads "whole Tesla". After accepting, he sees each passenger's first name, drop zone and the cash fare to collect.

Fare model

$\text{distanceCharge} = \text{distance_km} \times 25 \text{ Tk}$

$\text{poolDiscount} = 25\% \text{ of distanceCharge}$ (only if the ride starts with 2 or more separate bookings)

$\text{farePerSeat} = 40 \text{ Tk (base)} + \text{distanceCharge} - \text{poolDiscount}$

$\text{passengerFare} = \text{farePerSeat} \times \text{seats booked}$

The discount depends on whether the ride is shared, not on how many people share it. It is rounded down to a whole poysa. The server never accepts a fare from the client and recalculates it from the zone distance table. Two endpoints serve the screens: one takes zones, and one takes coordinates and returns the exact zones a booking would get, so the fare on screen and the stored fare cannot disagree.

	Nusrat, Banani to Mohakhali, 3.0 km	Rafiq, Banani to Gulshan 1, 2.0 km
Base	40.00	40.00
Distance charge	3.0 x 25 = 75.00	2.0 x 25 = 50.00
Pool discount, 25%	minus 18.75	minus 12.50
Pooled fare	Tk 96.25 (9625 poysha)	Tk 77.50 (7750 poysha)
Fare riding alone	Tk 115.00 (11500 poysha)	Tk 90.00

In scenario E, Rafiq pays $40 + 50.00$ minus $12.50 = \text{Tk } 77.50$, Nusrat pays Tk 96.25, and Shirin, at 4.0 km, pays $40 + 100.00$ minus $25.00 = \text{Tk } 115.00$. Jashim collects Tk 288.75 in cash. Nusrat's and Rafiq's fares are the same as in scenario A.

Private hire

The private fare is $(\text{base} + \text{distance charge}) \times 3$, with no pool discount. For Nusrat in Bullet from Banani to Mohakhali that is $(40 + 75) \times 3 = \text{Tk } 345.00$, or 34500 poysha. Version 6 multiplied by the accepting vehicle's real seat count. The final design always uses 3 seats, because the passenger sees the quote before any vehicle is chosen, and a driver with a six-seat Tesla would otherwise have charged Tk 690 after a quote of Tk 345. The multiplier is 3 because a smaller number such as 1.5 would make private cheaper than sharing. Two friends booking two shared seats pay Tk 230 for the trip above, and a 1.5 multiplier would price a private ride at Tk 172.50, so nobody would book shared seats. A private ride is picked up at the stand like any other, so there is no door-pickup surcharge. When a private request is accepted the ride's seats taken are set to its capacity, so the ordinary capacity check blocks everyone else with no separate locking code.

When the fare is decided

Moment	What the passenger sees
Choosing a trip	The solo estimate (Tk 115.00) and "as low as Tk 96.25 if shared".
Matched, others in the car	The estimate updates to the pooled price.
Someone cancels or is a no-show before the start	Her price does not go up. Once she has been shown a pooled price, that is her ceiling.
The trip starts	The fare is locked at the lower of the calculated fare and her ceiling, and it is stored with its breakdown. It never changes during the ride.
Dropped off early	No change.

The ceiling is the column fare_cap_poysha. It starts at the solo fare when she is matched. Whenever someone joins her ride, every booking in the ride takes the lower of its ceiling and its pooled fare. A private booking's ceiling is set to the quoted private price when the request is created. At the start of the trip the final fare is the lower of the calculated fare and the ceiling. If Rafiq joins Nusrat's ride and then does not show, Nusrat still pays Tk 96.25 and the driver absorbs the Tk 18.75 difference. Without the ceiling she would see Tk 96.25, sit in the vehicle and be charged Tk 115.00 because someone else did not appear.

The breakdown stored at the start of the trip lets the receipt show the base fare, the distance charge, the pool discount and any ceiling that lowered the price, so the evaluator can check the total by hand. Payment is cash, paid to the driver at drop-off. The design's payment_method

column was not built, and the TeslaPay wallet from version 1 was dropped in version 2 to make time for the safety features.

Data model

The database has nine tables. The first table below is the entity relationship diagram in text form. It reads from a parent table to the tables that point at it. The second table is the data dictionary. The full column list is in `api/src/db/schema.js`, and the SQL is in the nine migration files in `api/drizzle`.

users (driver) 1 → 0..1 vehicles
 vehicles 1 → many rides
 rides 1 → many ride_requests (the pool: bookings with the same ride_id)
 users (passenger) 1 → many ride_requests
 rides 1 → many ride_events, ride_requests 1 → many ride_events, users 1 → many ride_events (actor)
 zones 1 → many places, vehicles (current area), rides, and ride_requests (pickup zone and drop zone)
 zones 1 → many zone_distances (from zone and to zone)
 places 1 → many rides and ride_requests (pickup stand)
 otp_codes has no relationship. It is matched to users by phone number.

Relationships

Parent	Child	Cardinality	Foreign key
users	vehicles	1 to 0..1	vehicles.driver_id, unique, so one Tesla per driver
users	rides	1 to many	rides.driver_id
users	ride_requests	1 to many	ride_requests.passenger_id
users	ride_events	1 to many	ride_events.actor_id
vehicles	rides	1 to many	rides.vehicle_id
rides	ride_requests	1 to many	ride_requests.ride_id, null while the booking is waiting
rides	ride_events	1 to many	ride_events.ride_id
ride_requests	ride_events	1 to many	ride_events.request_id
zones	places	1 to many	places.zone_id
zones	vehicles	1 to many	vehicles.current_zone_id
zones	rides	1 to many	rides.zone_id
zones	ride_requests	1 to many, twice	ride_requests.pickup_zone_id and drop_zone_id
zones	zone_distances	1 to many, twice	zone_distances.from_zone_id and to_zone_id
places	rides, ride_requests	1 to many	rides.pickup_stand_id

Parent	Child	Cardinality	Foreign key
			and ride_requests.pickup_st and_id

Data dictionary

Table	Column	Type	Rules
users	id	uuid	Primary key.
	name, password_hash	text	Password stored only as a bcrypt hash.
	phone	text	Unique. Normalised to one format (+880...) before any lookup.
	role	enum	PASSENGER or DRIVER.
	gender	enum	FEMALE, MALE or UNDISCLOSED. Set at signup and never editable.
	phone_verified_at	timestampz	Empty until the one-time code is confirmed.
	nid_hash, nid_last4, nid_verified_at	text, text, timestampz	nid_hash is unique and is an HMAC of the NID, so one NID allows one account. Only the last four digits are kept readable.
otp_codes	id, phone, code_hash	uuid, text, text	The code is stored hashed. Old codes for the same phone are invalidated when a new one is sent.
	expires_at, attempts, created_at	timestampz, integer, timestampz	Expires after five minutes. At most five attempts.
zones	id, name, center_lat, center_lng	uuid, text, double	name is unique.
zone_distances	from_zone_id, to_zone_id, distance_km	uuid, uuid, double	The primary key is the pair, so there are no duplicates. Stored in both directions.
places	id, name, kind, lat, lng, zone_id	uuid, text, enum, double, uuid	kind is STAND or LANDMARK.
vehicles	id, driver_id, name, registration_no	uuid, uuid, text, text	driver_id and registration_no are both

Table	Column	Type	Rules
			unique.
	capacity	smallint	CHECK between 1 and 6.
	is_online, current_zone_id, last_seen_at	boolean, uuid, timestampz	The area is empty until the driver goes online. last_seen_at feeds the online count.
rides	id, vehicle_id, driver_id, pickup_stand_id, zone_id	uuid	One row is one car trip, the pool. Only one active ride per vehicle, by a partial unique index.
	status	enum	OPEN, ARRIVED, STARTED, COMPLETED or CANCELLED.
	capacity, seats_taken	smallint	capacity is copied from the vehicle. CHECK that seats_taken is between 0 and capacity.
	is_private	boolean	Set by the first accepted booking. Women-only is not stored here.
	arrived_at, started_at, completed_at, created_at	timestampz	Drive the timers.
ride_requests	id, passenger_id, ride_id	uuid	One row is one booking. ride_id is empty while waiting.
	status	enum	REQUESTED, MATCHED, DRIVER_ARRIVED, IN_PROGRESS, COMPLETED, CANCELLED, EXPIRED or NO_SHOW.
	seats, ride_type, women_only	smallint, enum, boolean	seats has a CHECK between 1 and 6. ride_type is SHARED or PRIVATE.
	pickup_stand_id, pickup_zone_id, drop_zone_id	uuid	Every booking stores the stand and its zone, and the drop zone.

Table	Column	Type	Rules
	pickup_lat, pickup_lng, drop_lat, drop_lng	double	The pickup point is always empty now, since private rides use the stand too. The drop point is stored so the map can draw the route.
	fare_cap_poysha, fare_poysha, fare_breakdown	integer, integer, jsonb	The price ceiling, the locked fare and the stored working, all in poysha.
	idempotency_key, body_hash	text	Unique together with passenger_id. The hash detects the same key reused with a different request.
	queued_at, boarded_at, dropped_at, created_at	timestampz	queued_at resets each time the booking returns to waiting, and expiry counts from it.
ride_events	id, ride_id, request_id, actor_id	uuid	Append-only. The code only inserts.
	type, from_status, to_status, details, created_at	text, text, text, jsonb, timestampz	Written in the same transaction as the change it describes.

Constraints and indexes

The database enforces these rules regardless of what the application does: capacity between 1 and 6, seats taken between 0 and capacity, booking seats between 1 and 6, unique phone, unique NID hash, unique registration number, one vehicle per driver, one active ride per vehicle, one active booking per passenger, and one booking per passenger and idempotency key. The two "one active" rules are partial unique indexes, since "at most one active row" applies only while the status is OPEN, ARRIVED or STARTED for a ride, or REQUESTED, MATCHED, DRIVER_ARRIVED or IN_PROGRESS for a booking. Finished rides and bookings do not count against the limit. These constraints also produce readable errors. The error handler maps Postgres codes 23505 (unique) and 23514 (check) to a 409 response.

Design choices in the schema

- Pool membership is a column on ride_requests and has no table of its own. A booking belongs to at most one ride, so ride_id on ride_requests is enough, and "who is in ride 42" is every booking with that ride_id. A join table is for many-to-many, which cannot happen here. If bookings ever hopped between rides, the history already lives in ride_events.
- Capacity is copied into the ride. A CHECK constraint can see only its own row, so to guard "seats taken never exceeds capacity" both values must be on the rides row. If a vehicle were edited mid-trip, the ride would keep the rule it started with.

- seats_taken is a stored counter. Adding up bookings is always correct, but the database cannot put a CHECK on a sum. A stored counter can be guarded and locked, at the cost of updating it in the same transaction as every accept and cancel. A test checks that it equals the sum of the seats of active bookings.
- Identifiers are random UUIDs, which are hard to guess and do not reveal how many rides exist. Ownership is still checked on every request, so this is an extra layer.
- Money is stored as integer poysa, so there are no rounding drifts.

Migrations and seed data

The migrations, generated by drizzle-kit and renamed for readability, are in order: zones and places, users, vehicles, OTP and identity columns, the vehicle's last-seen time, rides and requests, fare lock and drop-off time, the drop point, and the fare breakdown. Docker starts the API with a script that runs the migrations, then seeds Dhaka (12 zones, 66 distances, 42 places), the two drivers and their vehicles, and the three passengers. The seed is safe to run twice, because a second start of the containers must not crash on a unique rule, and it updates existing zones, places and distances rather than skipping them.

Ride lifecycle

The brief suggests one chain of statuses. The design improves on it with two linked chains, one for the car trip and one for each passenger's booking, because in a pool the ride and a booking are different things. If Rafiq cancels, the ride continues while his booking is cancelled, and one status cannot describe both. The brief also says each passenger must see their own status.

Booking status	REQUESTED → MATCHED → DRIVER_ARRIVED → IN_PROGRESS → COMPLETED. Other endings: CANCELLED, EXPIRED (no driver in 15 minutes), NO_SHOW (absent five minutes after arrival). A driver who cancels the ride, or who never arrives, sends the booking back to REQUESTED.
Ride status	OPEN (taking passengers) → ARRIVED (group locked) → STARTED (fares locked) → COMPLETED (last drop-off). The ride can end as CANCELLED before it starts.

How the two chains move together

Action (who)	Ride becomes	Bookings become	Allowed when
Passenger requests	no change	REQUESTED	Phone verified, no other active booking.
Driver accepts the first request	OPEN (created)	That booking: MATCHED	Driver online with a vehicle, no active ride.
Driver accepts another	stays OPEN	That booking: MATCHED	Ride OPEN and the request fits rules R1 to R6.
Driver presses arrived	ARRIVED	All: DRIVER_ARRIVED	Ride OPEN with at least one booking.
Driver ticks boarded	stays ARRIVED	That booking gets boarded_at	Ride ARRIVED.
Driver marks a no-show	stays ARRIVED, seats freed, auto-cancelled if	That booking: NO_SHOW	Five minutes since arrival and not boarded.

Action (who)	Ride becomes	Bookings become	Allowed when
	nobody is left		
Driver starts	STARTED	Boarded: IN_PROGRESS with fares locked. Not boarded: back to REQUESTED ("driver left without you").	Ride ARRIVED with at least one boarded passenger.
Driver drops one passenger	COMPLETED when the last one is dropped	That booking: COMPLETED	Ride STARTED and the booking IN_PROGRESS.
Passenger cancels	stays, seats freed, auto-cancelled if it was the last booking	Theirs: CANCELLED	Booking REQUESTED or MATCHED, or DRIVER_ARRIVED once the driver has failed to start for 15 minutes.
Driver cancels the ride	CANCELLED, seats set to 0	All back to REQUESTED, not cancelled	Ride OPEN or ARRIVED.
Driver never arrives	CANCELLED by the system after 15 minutes	All back to REQUESTED with a fresh queue time	Ride still OPEN 15 minutes after the first accept.
Nobody accepts	none	EXPIRED	15 minutes since queued_at.
Ride STARTED for over 3 hours	COMPLETED, recorded as auto-closed	Remaining: COMPLETED	Checked on read, so a vanished driver cannot trap passengers.

Rules that follow

- A driver cannot remove anyone. Before accepting he chooses freely. After accepting he can cancel the whole ride before it starts, or mark a no-show after five minutes. He also cannot re-accept a booking he cancelled, which closes the route of cancelling a whole ride and re-accepting everyone except one person.
- Passengers cannot cancel after the driver arrives, because the driver is standing there, unless he has not started within 15 minutes. A production build would add a cancellation fee.
- If a driver cancels, the passengers are not penalised. Their bookings return to the waiting list with a fresh queue time, so another driver can take them.
- Each passenger is dropped at their own stop, and there is no separate complete-ride button. The ride completes when the last passenger is dropped.
- A driver may leave before the vehicle is full. He presses arrived, then start, with any number of boarded passengers, and anyone not boarded returns to the waiting list.
- A driver cannot go offline or change his area during an active ride, and cannot go online without a vehicle and a verified phone. The API enforces this as well as the screen, because an account may be open on several devices.
- A driver cannot cancel a started ride. Cancelling mid-ride is not an app action, and a

breakdown would need partial fares, so it is a known limitation.

Time rules

Every timer is lazy. Instead of a background job, the server checks the deadline whenever the affected data is read, and updates the status then. The result is the same and no scheduler is needed. Each rule is a named constant in the code.

Rule	Value
A waiting request expires without a driver	15 minutes from queued_at
A driver who accepted must press arrived	15 minutes from the first accept
A driver may mark a no-show	5 minutes after arrival
A passenger may cancel after arrival if the trip has not started	15 minutes after arrival
A started ride is auto-completed	3 hours after the start
A driver counts as online	Seen within 5 minutes
A one-time code lives	5 minutes, 5 attempts, 3 sends per 15 minutes per phone
A login session lives	7 days, renewed on each request
Live-update heartbeat and polling fallback	Heartbeat every 25 seconds. Polling every 5 seconds after 30 seconds of silence.

The transition table

All allowed moves live in one module, `rideStateMachine.js`, as two tables that name the actor allowed to make each move: the driver, the passenger, the system, or the demo tooling. Every service asks it before writing a status. The actual write is also a conditional update, such as changing a ride to `STARTED` only where its status is still `ARRIVED`. If the driver double-taps `start`, the second tap changes no rows and receives a polite 409. Every successful change also inserts a `ride_events` row in the same transaction, so the history cannot disagree with the data.

Concurrency and data consistency

Bullet has one seat left. Nusrat and Shirin both try to take it at nearly the same instant, and each screen shows "1 seat free". A naive server reads the ride (2 of 3 taken, so one is free), then writes 3. If both requests read before either writes, both succeed and Bullet carries four people, and the counter shows a wrong value.

Three layers of protection

Layer	What it does
1. Lock the ride row	The accept runs in a transaction that begins with <code>SELECT ... FROM rides WHERE id = ... FOR UPDATE</code> . The second request waits at that line until the first commits, then reads the fresh count and is refused.
2. Conditional updates	The booking is claimed only <code>WHERE status = 'REQUESTED'</code> , and seats are added only

Layer	What it does
	WHERE seats_taken + n is no more than capacity. If a condition is false at that instant, zero rows change and the transaction rolls back.
3. Database CHECK	CHECK (seats_taken between 0 and capacity). Even a bug in the code cannot save four people in a three-seat vehicle, because Postgres refuses the row.

The accept transaction

Step	Action	If it fails
1	Clean up expired requests and abandoned rides, then begin the transaction and lock the driver's open ride row.	The transaction does not start.
2	Claim the booking: set MATCHED and the ride id only where the booking is still REQUESTED and queued within 15 minutes. Refuse a booking this driver cancelled earlier.	Zero rows: 409 REQUEST_UNAVAILABLE.
3	Check the driver is online in the right area, then run pooling rules R1 to R6 as a pure function on the fresh data.	409 DOES_NOT_FIT with every reason.
4	Add the seats only if they fit. For a private booking set seats taken to capacity, only where seats taken is 0.	Zero rows: roll back, 409.
5	Update the price ceilings of every booking in the ride, and insert the ride_events row.	Any error rolls everything back.
6	Commit, then send nudges to the driver and everyone in the ride.	No nudge is sent for a rolled-back change.

The losing request sees its booking unchanged and receives a specific 409 conflict, and the driver's screen shows a friendly message and refreshes. The first accept of a driver's first ride has no ride row to lock yet, and there the partial unique index allowing one active ride per vehicle decides the winner.

Other races

- Two drivers and one passenger. Jashim and Mokbul both accept Nusrat. Only one conditional update on her booking can succeed. The second finds her already MATCHED, changes zero rows and receives 409 REQUEST_UNAVAILABLE.

- A double tap by a passenger. The screen creates one random idempotency key when the form opens and sends it with the request. The database has a unique rule on the passenger and key, so the second copy returns the booking that already exists, and the same key with a different request is refused with 422. A key longer than 100 characters is refused with 400. The rule of one active booking per passenger is a second safety net.
- Two tabs of one account. The unique rules reject the second, and the error handler turns the database code into a readable 409.
- Six-digit code guessing. The attempt counter is one atomic update that increments only while attempts are below five, so twenty parallel guesses cannot bypass the limit. A test fires twenty guesses and expects exactly five wrong answers and fifteen refusals.

Every transaction that touches a ride locks the ride row first and then its bookings. If one transaction locked a booking first and another locked the ride first, each could wait for the other forever. With one fixed order that cannot happen, and a deadlock error (code 40P01) is retried once as a safety net.

What the tests show

The concurrency test fires two accepts for the last seat with `Promise.all`, expects exactly one 200 and one 409, and checks that seats taken is 3, not 4. It repeats the race to catch timing-dependent failures. A sabotage check was also run: removing the row lock from the accept function did not fail any test, because the conditional update and the CHECK constraint still let only one request win. The lock is therefore one layer of three. A second sabotage, running the two accepts one after the other in the demo, did fail the overlap tests, which shows the tests can tell a real race from a fake one.

The Seat Race demonstration

The Seat Race page, reachable from the login screen, sets up the last-seat scenario and fires both real accept calls together. It shows the outcome of each contender, the time each transaction began to the millisecond (taken inside the transaction through a hook, after the connection is acquired), the database connection each used, whether the transactions overlapped, the final seat count and the events the database recorded. A typical run shows the two transactions starting about 0.05 ms apart on different connections, one winner, seats 3 of 3, and exactly two "matched" events. A second race, two drivers and one request, sits beside it. Both are available only when `DEMO_MODE` is on and a demo key of at least 32 characters is supplied. The database connection pool has at least two connections, since with one connection the two accepts would run one after the other and prove nothing.

At larger scale

The same approach holds, because locks are per ride and a hundred thousand drivers never wait on each other. Only two people contending for the same Tesla do. At scale, add lock timeouts, retry with backoff on 409, and metrics for lock waits. If matching becomes automatic and very busy, move it to one matcher per city zone so each zone's decisions happen one at a time in memory.

API design

The API is REST with JSON. Status changes are action endpoints such as POST /driver/ride/start, each with its own permission check and its own allowed-from status, instead of one generic status endpoint with a pile of conditions. All paths are reached through the /api rewrite.

Group	Who	Endpoints
Authentication	Anyone, then logged in	POST /auth/signup, /auth/login, /auth/logout; GET /auth/me; POST /auth/otp/send and /auth/otp/verify
Places and fares	Logged in	GET /places?q=, GET /zones, GET /zones/:zoneId/online-count (with an optional standId), POST /places/nearest-stand, POST /fares/estimate, POST /fares/quote
Passenger	Verified passenger, owner only	POST /requests (with an Idempotency-Key header); GET /requests; GET /requests/:id; POST /requests/:id/cancel; GET /requests/:id/timeline; GET /requests/:id/ride
Driver vehicle and status	Driver	POST and GET /driver/vehicle; POST /driver/online and /driver/offline
Driver pooling and trip	Driver	GET /driver/requests; POST /driver/ride/requests/:id/accept; POST /driver/ride/arrived; POST /driver/ride/requests/:id/board; POST /driver/ride/requests/:id/no-show; POST /driver/ride/start; POST /driver/ride/requests/:id/drop; POST /driver/ride/cancel; GET /driver/ride; GET /driver/history
Live updates	Logged in	GET /events/stream
Health	Anyone	GET /health, which returns 503 when the database is unreachable
Demo	DEMO_MODE and the X-Demo-Key header	POST /dev/scenario/:name with reset, seat-race and two-drivers

Errors

Every error has one shape: an object with a code and a message, for example SEATS_UNAVAILABLE with "Only 1 seat left in Bullet, this request needs 2." A custom AppError is thrown from services, and one error-handling middleware converts it.

Status	When
400	Invalid data, including the field that failed, malformed ids, a bad idempotency key, and locations outside the service area.
401	Not logged in, or a wrong demo key.
403	Wrong role, or PHONE_NOT_VERIFIED.
404	Not found, or not yours. Rafiq requesting Nusrat's booking receives 404 instead of 403, so he cannot learn that it exists.
409	Conflict with current state: wrong status, seat taken, request already accepted, DOES_NOT_FIT with reasons, NO_SHOW_TOO_EARLY, RIDE_IN_PROGRESS, or a duplicate.
422	The same idempotency key sent with a different request.
429	Too many attempts.
500, 503	A bug, logged with the request ID, or an unreachable database.

Security and privacy

Threat	Protection
Stolen passwords	bcrypt hashes. Plain passwords are never stored or logged.
Token theft by scripts	The JWT is in an httpOnly cookie, Secure in production and SameSite Lax, valid for seven days and renewed on each request.
Forged tokens and weak secrets	The environment is validated with Zod at start-up. In production the API refuses to start if JWT_SECRET, NID_PEPPER or DEMO_KEY is missing, shorter than 32 characters, or the example value.
Fake or duplicate accounts	Phone numbers are normalised to one format. The NID is stored as an HMAC keyed by a separate NID_PEPPER, unique across accounts, and only the last four digits are readable. Gender is locked after signup.
Secrets in logs	Request bodies are never logged, and password, code and nid are redacted from anything that is logged.
One user touching another's ride	Every query includes the owner. Driver actions

Threat	Protection
	include the driver id. Other people's resources return 404. This is tested.
Passenger calling driver actions	requireRole. Verification and vehicle status are read from the database, not from the token.
Fake prices from the browser	The server never accepts a fare and recalculates it from zones.
Bad input and injection	Zod on every body, query and parameter. Drizzle uses parameterised queries. LIKE wildcards are escaped.
Password guessing	Login is rate limited by normalised phone number, counting failed attempts only, so different spellings of a number share one limit and successful logins cannot lock an account.
Code guessing and SMS abuse	Codes come from a secure random generator and are stored hashed. Expiry is five minutes, attempts are limited to five with an atomic counter, and sends are limited to three per phone per 15 minutes.
Unsafe pooling	The women-only rule is enforced on the server inside the accept transaction, and co-riders see first names only.
Home addresses	No home location is stored. Every ride, private or shared, is picked up at a public stand.
Header and cross-site attacks	Helmet adds the security headers. No CORS package or headers exist, because the browser only calls its own origin, which is the strictest setting, and a test locks it in. Cookies use SameSite Lax.
Demo routes on a public server	They return 404 unless DEMO_MODE is on and 401 without the demo key. Both sides are hashed and compared in constant time, and the routes are rate limited. The key is shared privately and is not in the README. The reset touches only the demo cast.
Leaked secrets	.env is ignored by git and only .env.example is committed. Production secrets live in the hosts' dashboards and were generated with openssl rand.

Who can see what

Viewer	Sees	Never sees
Passenger	Co-riders' first names and how many there are, the driver's name, the vehicle name and the registration number, her own fare, receipt and timeline.	Co-riders' phones or fares, or their exact points.

Viewer	Sees	Never sees
Driver, before accepting	Stand, drop zone, seats, shared or private, women-only, wait time and the fares he would collect.	Anything about people he has not accepted beyond that.
Driver, after accepting	His passengers' names, drop zones, seats, ride type and the cash fare to collect from each.	Passengers' phones and passwords, or rides that are not his.

The design also called for a "phone verified" badge beside each co-rider. It was not built.

Live updates

Live updates use Server-Sent Events on one endpoint. The server keeps an in-memory map from user id to that user's open connections, so several tabs of one account all update. After a transaction commits, a single notify function sends a small message to the users the change affects. A table in the realtime module lists, for each kind of event, which users are nudged, so a cancellation, an expiry or an auto-close cannot silently notify nobody.

The message is a nudge that carries only identifiers, such as the ride that changed. It never contains a fare or a name. The browser receives it and marks its cached data as stale, and TanStack Query re-fetches through the normal API, which applies the privacy rules. The privacy logic therefore lives in one place, the API, and a live update cannot leak what an ordinary request would hide. Nudges are sent only after the commit, because sending inside a transaction would let a phone re-fetch before the data was saved and show the old status.

- The server sends a heartbeat every 25 seconds. It is a real message, so the client can tell a healthy stream from a dead one, and it refreshes the driver's last-seen time.
- If nothing arrives for 30 seconds the client polls every five seconds until the stream returns, and it re-fetches once on reconnect in case a nudge was missed.
- The client has its own reconnect with backoff, from 2 up to 30 seconds. The browser gives up for good when a proxy answers a reconnect with a 500, which happened during testing while the API was down.
- Streams held open through the deployed Vercel and Render setup for two minutes were not cut, and events reached both sides within a fraction of a second.

The in-memory connection list works with one server. With two servers a passenger connected to one might miss an update made on the other, and the fix is Postgres LISTEN and NOTIFY, then Redis publish and subscribe, behind the same notify function.

Frontend

The frontend is a Next.js App Router application. It holds no business rules. It fetches everything on the client through TanStack Query, sends every request through one API wrapper that turns failures into errors carrying the server's status, code and message, and shows the result. A network failure becomes a readable OFFLINE message instead of a raw browser error.

Screens

Version 6 planned separate pages for a booking, history and the driver's vehicle. The build merged them into two working pages, one per role, and kept the map behind all of them.

Route	Who	Contents
/login	Everyone	Phone and password, one-tap demo accounts for the cast, and a For reviewers row that opens the Seat Race.
/signup, /verify	New users	Name, phone, password, role, NID and gender (locked, with a warning). Then the one-time code step, where the code is shown on screen in demo mode.
/ride	Passenger	Search for pickup and drop-off with the nearest stand, walk time and the count of Teslas online. Seats, a Share or Private switch, the women-only switch for female accounts, and the solo, pooled and private fares. Then the live booking: a status stepper, driver, car and co-riders, timeline and cancel button. Then the receipt and past rides.
/driver	Driver	The add-your-Tesla form, the online switch with an area picker, waiting requests with fit reasons and fares, the current ride with seat icons, the arrival countdown, boarding ticks, the no-show countdown, the drop-off list in suggested order with each cash fare, and recent rides.
/seat-race	Public	The concurrency demonstration, which asks for the demo key.

The map

The first map used Leaflet in a card on the ride screen. It was replaced by MapLibre with OpenFreeMap vector tiles, recoloured to match the app, with one full-screen map behind every screen. On a phone the content sits in a draggable bottom sheet, and on a laptop it sits in a floating panel on the left. The map shows the route, the stand, the drop-off, a marker for the Tesla and, for the driver, the requests, the ride and the drop-offs. The map is for looking only. Pickup and drop-off come from search, since a tapped point had no name to show and the app has no geocoder.

Look and feel

The visual direction, set before the first component was written, is an Uber-style interface built on Apple's Human Interface Guidelines: the Apple system font stack, Apple's named text sizes, a grouped grey background with white cells, a black primary button, and Apple's accessible variants of green, red and blue for text. Cells and controls use a 12 px radius and sheets use 20 px. The app is light mode only, because a half-finished dark mode looks worse than none. A written list of twenty patterns that make an interface look generated, such as gradient headings, emoji in headings, glassmorphism and untouched component-library defaults, was kept as a checklist. Buttons show a pressed state instead of a hover fade.

States and behaviour

- Every screen handles four states: loading (skeleton cards), empty (for example "No one is waiting in Banani right now"), error (a message and a retry) and success.
- When the server is asleep, a card reads "Waking up" and explains that the page fills in by itself. Requests retry for up to a minute, and the card disappears when the server answers.
- The signed-in name and role appear at the top right of every screen, so four browser windows can be told apart during testing. Logging out clears the previous person's cached data but keeps the entry for the current user, which fixed a bug where the old name stayed after switching accounts.
- Deadlines shown on screen, such as the driver's arrival countdown and the no-show wait, come from the server, so a phone with a wrong clock cannot show a rule as satisfied when it is not.
- Timestamps are stored as `timestamp_tz` and formatted in Asia/Dhaka on the client, since the host runs in UTC.
- A fresh idempotency key is created for each distinct request, and reused if the passenger taps the button again on the same request.
- The receipt shows the route, the drop-off time, the ride type, driver, car and registration number, and the fare working: base fare, distance charge, pool discount and any ceiling that lowered the price.

Testing

The brief asks for meaningful tests, not coverage numbers. The tests cover the rules that would hurt real people if they broke, and they run against a real PostgreSQL database, because the riskiest code is database behaviour that a fake cannot prove.

Approach

- Vitest runs the tests and Supertest sends real HTTP requests to the Express app without opening a port, so the endpoints are exercised as the frontend uses them.
- A separate database named `seatase_test` is created by the global setup and migrated and seeded on the first run, so tests never touch the demo data. Test files run one at a time, because they share one database and the same cast, and each file resets its own tables. Test servers bind to 127.0.0.1 so random ports cannot collide.

- The cast is the test data. Nusrat's and Rafiq's pooled fares appear as fixed numbers in the tests, in the README and in the demo.
- Each rule in the review-fix phase was written as a failing test first, then fixed, then checked with a sabotage step that reintroduces the bug and confirms the test fails.
- A Postman collection of 52 requests with 61 assertions covers the whole story, from signup to a completed pooled ride and the Seat Race. It passed with no failures against a freshly cloned copy.
- There is no large frontend test suite. The risk lives in the backend, and the screens were checked by hand in four browser windows and in headless Chrome at phone size. A Playwright suite is future work.

Coverage of the brief's requirements

Requirement	Test
Capacity can never be exceeded	Fill Bullet with real bookings and try one more accept: refused. Setting seats taken to 4 directly is rejected by the database CHECK.
Invalid state transitions are rejected	The transition tables are checked cell by cell, and the endpoints are called in the wrong order: start a ride that has not arrived, complete one that has not started.
Pooled fares are correct	Nusrat 9625 poysha, Rafiq 7750, Nusrat alone 11500, Nusrat private 34500. Scenario E gives 7750, 9625 and 11500.
Users cannot touch other users' rides	Rafiq reading or cancelling Nusrat's booking gets 404. Mokbul starting Jashim's ride gets 404. A passenger calling a driver endpoint gets 403.
Cancellation rules hold	Cancelling while matched frees the seat. After the driver arrives, cancelling is refused. A driver's cancel returns every passenger to waiting and clears the boarded tick. A driver cannot re-accept a booking he cancelled.
Concurrent requests cannot break capacity	Two accepts for the last seat fired together give one 200 and one 409 with seats 3 of 3. Two drivers accepting one booking give one winner. Both are repeated.
Idempotency	The same key twice gives the same booking and one row. The same key with a different body gives 422. A 3,000-character random key gives 400.
Women-only and private	Shirin's request cannot join Bullet with Rafiq on board. A women-only request joins an all-female ride. A male request cannot join a women-only ride. A private accept sets seats to capacity and blocks a later shared accept.
Identity	An unverified user cannot request (403). An

Requirement	Test
Capacity can never be exceeded	Fill Bullet with real bookings and try one more accept: refused. Setting seats taken to 4 directly is rejected by the database CHECK.
Invalid state transitions are rejected	The transition tables are checked cell by cell, and the endpoints are called in the wrong order: start a ride that has not arrived, complete one that has not started.
Pooled fares are correct	Nusrat 9625 poysha, Rafiq 7750, Nusrat alone 11500, Nusrat private 34500. Scenario E gives 7750, 9625 and 11500.
	expired code and a sixth wrong attempt are rejected, including twenty parallel guesses. A repeated NID gives 409. A gender change is rejected.
Trip lifecycle	A no-show before five minutes gives 409. Dropping Nusrat leaves the ride STARTED, dropping the last passenger completes it and moves the driver's area. Starting with an unboarded passenger returns her to waiting.
Price ceiling	Rafiq joins and then does not show. Nusrat pays 9625, not 11500. A private fare never exceeds its quote, even in a six-seat vehicle.
Stuck states	A driver who never arrives, a passenger after 15 idle minutes at the stand, and a ride started three hours ago are all resolved on read.
Geography and seed data	A pin at Banani Road 11 snaps to its stand with the right walk distance, a pin in Chattogram is rejected, every one of the 66 zone pairs has a distance, and the Gulshan 2 correction reaches an existing database.
Security behaviour	Helmet headers are present, a request from another origin gets no CORS header, the cookie is httpOnly and SameSite Lax, demo routes return 404 or 401, malformed ids return 400 and login limits apply across spellings of a phone number.

At release the suite held 281 tests in 27 files. The count grew with the build: 50 after the places and fares phase, 209 after the Seat Race, 218 after the abandoned-ride rule, 223 after the map redesign and 281 after the review fixes. Every count was checked on the merged master before pushing.

Deployment and operations

Running locally

The command `docker compose up -d --build` starts three services: the database, the API and the web app. The database has a health check, the API waits for it and runs a start script that applies the migrations, seeds the data and then starts the server, and the web app waits for the API. The script uses `exec` so that Node receives the stop signal from Docker, and it stops if a migration fails and does not start a server on a half-built database. Data survives restarts in a named volume, and every published port can be changed with `DB_PORT`, `API_PORT` and `WEB_PORT` when a machine already runs Postgres on 5432. A fresh clone was tested the way an evaluator would run it and became healthy in about 17 seconds with a warm cache. Tests run on the host, not in a container. They need Node 22 and the database from `docker compose up -d db`, then `npm ci` and `npm test`, with `DB_PORT` set if the port was changed.

Environment variables

Variable	Purpose
<code>DATABASE_URL</code>	Postgres connection string.
<code>JWT_SECRET</code>	Signs login tokens. Production needs at least 32 characters and refuses the example value.
<code>NID_PEPPER</code>	Key for the NID fingerprint. It is set once and never changed, because old fingerprints cannot be recomputed with a new key.
<code>DEMO_MODE</code> , <code>DEMO_KEY</code>	<code>DEMO_MODE</code> shows the one-time code on screen and enables the demo routes. <code>DEMO_KEY</code> must be at least 32 characters when demo mode is on in production.
<code>COOKIE_SECURE</code>	False on localhost and true in production, because a Secure cookie can fail on <code>http://localhost</code> in some browsers.
<code>SSE_HEARTBEAT_MS</code> , <code>LOG_LEVEL</code> , <code>PORT</code>	Heartbeat interval (25000 by default), log level and API port.
<code>DB_PORT</code> , <code>API_PORT</code> , <code>WEB_PORT</code>	Published ports for Docker Compose.
<code>API_URL</code>	Where the Next.js rewrite sends <code>/api/*</code> calls. Read at build time.

The rules that decide time and distance (3 km, 15 minutes, 5 minutes) are constants in the code, and no environment variable exists for them. The change-a-rule-live exercise is a one-line edit of `MAX_DROP_KM`.

Free hosting

Piece	Where	Notes
Web	Vercel, root directory web	<code>API_URL</code> points at the Render URL. Vercel installs from inside the web folder, which exposed a missing dependency on the shared package, fixed by declaring it in both web and API.
API	Render free web service, built	Health check on <code>/health</code> . It runs

Piece	Where	Notes
	from api/Dockerfile, Singapore region	migrations and seeds on start. It sleeps after about 15 minutes idle.
Database	Neon free Postgres 16, Singapore, direct connection string	Render's free Postgres is deleted after 30 days. The pooled connection string is avoided because of prepared-statement problems.
Uptime	A free UptimeRobot monitor on /api/health every five minutes	Keeps the free API awake during evaluation.

A cold start was measured at 45.6 seconds on the live site. The application handles it by retrying and by showing the Waking up card, so a first visitor sees an explanation instead of an error. The monitor removes the wait in normal use. The release was then checked end to end on the live site: a full lifecycle by script and in a real browser, with events delivered to both sides within about three seconds of each action, the receipt breakdown, the driver's countdowns and the online count all as designed. The demo data was cleaned afterwards.

Secrets

Every local value is a placeholder on purpose. `.env.example` is committed with fake values and `.env` is ignored by git. Production secrets exist in one place only, the host's environment settings, generated with `openssl rand -base64 48`. The API refuses to start in production if any of them is missing, too short or the example value, so a running service proves the real values are set. `NID_PEPPER` is kept in a password manager because it can never change.

Git workflow

The repository has the long-lived branches `master`, `pre-release` and `release/v1.0.0`, and a `feature/*` branch for each piece of work. Fixes made after the build used `fix/*` branches, which is a documented variation on the brief's naming. Each branch holds one logical change with incremental commits, and is merged into `master` with `git merge --no-ff` and never squashed, so the merge appears in the history. Commit messages follow `type(scope): description`, for example `feat(pool): lock ride row when accepting a request`. Most messages were written with the commit-message shortcut in VS Code, which reads the staged changes and gave explanatory messages. Only `README.md` is committed as documentation. The development log stays local.

After the review fixes the full backend suite (281 of 281) and the web build passed on `master` at `125ab61`. `pre-release` was cut from `master`, `release/v1.0.0` from `pre-release`, and the annotated tag `v1.0.0` was pushed at the same commit.

Scaling to 1 million passengers and 100,000 drivers

The brief asks for reasoning, not for scaling tools inside the MVP, so none of this is built. The approach has two parts: build the MVP simple, with seams where a bigger part can be swapped in, and show on paper how it grows, with numbers.

The numbers first

Load	Assumption	Worst rush-hour minute
Booking requests and accepts	1M passengers, 15% ride on a weekday, which is 150k bookings a day. Half fall in four rush hours, about 5 a second, spiking 5 times.	About 25 requests and 25 accepts per second
Status changes	About five per booking.	About 125 per second
Screen reads	Every nudge makes a few screens re-fetch.	About 1,500 per second
Driver location pings	100k drivers, half online at rush hour, one ping every five seconds.	About 10,000 per second
Open live connections	50k drivers and about 60k waiting or riding passengers.	About 110,000

The bookings are tiny. One ordinary Postgres handles thousands of transactions a second, so the seat-lock design survives 1M users. What breaks is the 10,000 location pings a second, the 110,000 open connections and the reads. The plan is to scale what breaks and leave the rest alone.

Architecture at 1 million users

Passenger and driver apps → load balancer (spreads traffic, rate limits)		
API servers x N bookings and seat locks (stateless)	Realtime servers x N 110k open SSE connections	Location service 10k pings a second, latest position only
PostgreSQL primary (rides, money) and read replicas (history, driver feeds)	Redis publish and subscribe: "ride changed" reaches every realtime server	Redis geographic index, with zones becoming a grid of hexagon cells (H3)
Queue and workers for slow jobs: SMS one-time codes, receipts, analytics		

The API servers are copies of the same Express app. They work because the servers keep nothing between requests, since login is in the token and data is in Postgres. The realtime servers hold the SSE connections separately, so 110k open connections do not slow bookings. Redis publish and subscribe links them: Nusrat may be connected to realtime server 3 while Jashim's accept ran on API server 7, so server 7 publishes the change and every realtime server nudges its own users. The location service keeps only each driver's latest position, because ten thousand writes a second would choke Postgres and old positions are not needed. Rate limiting at the load balancer matters most for one-time codes, where bot sends cost real money. The Postgres primary takes all writes with the same locking code as the MVP, since locks are per ride, and read replicas take history and driver feeds.

Growing in stages

Users	What is added	Trigger
Demo	One API, one Postgres.	None
About 10k	Two or three API copies behind	API CPU stays above 70%

Users	What is added	Trigger
	a load balancer, a larger database, and Postgres LISTEN and NOTIFY to share nudges between servers. No new tools.	
About 100k	Redis for publish and subscribe and rate limits, a read replica, and a queue for SMS.	High database read load, and the SSE servers are full
About 1M	A location service with Redis geographic search and hexagon cells, a separate realtime tier, and matching split by zone.	Location writes choke the database
Several cities	A separate database for each city.	One primary cannot take the writes

Seams already in the MVP

MVP choice	Status and why it helps
Token in a cookie, no server memory	In place. Any number of API copies work from day one.
Seat lock per ride	In place. Drivers never wait on each other. Only people fighting over the same Tesla do.
One notify() function	In place. In-memory delivery can be swapped for LISTEN and NOTIFY and then Redis by changing one module.
One sendOtp() function	In place. A real provider and a queue can be added without touching authentication.
Zones as the matching unit	In place. Zones become the way work is split: one matcher per zone and one database per city.
Idempotency keys	In place. Retries are safe, which matters when networks fail constantly.
ride_events history	In place. It can feed analytics or an event stream without touching booking logic.
Indexes on hot queries	Partly. Only the indexes required by uniqueness rules exist. The lookup indexes for the driver's waiting list, a passenger's history, events by ride and one-time code lookup are not built.
Rules as configuration	Partly. The rules are named constants, and are not yet environment variables that could be tuned for each city without a redeploy.

When things fail

Failure	What happens
A server dies mid-accept	The transaction never committed, so Postgres

Failure	What happens
	rolls it back. No seat is half-booked.
Redis dies	Realtime falls back to polling from Postgres. It is slower but works, and the MVP already uses the same fallback.
The database primary dies	A replica is promoted, which managed databases do automatically.
The SMS provider is down	One-time-code jobs wait in the queue and retry with growing delays, and a circuit breaker stops the retries from hammering a dead provider.
A bad deployment	Rolling deployments update a few servers at a time, and migrations work with both old and new code.
Nobody notices a problem	Central logs with request IDs, metrics such as p95 latency, the 409 rate (seat fights) and lock waits, traces and alerts. None of this exists yet.

How the system was built

The build ran from the evening of 24 September to 30 September 2026, in the order the story happens: first the world, then the driver, then the passenger, then the ride. The whole backend came first, because that is where the grading is heaviest, and the screens followed in the same order. Each phase depended on the one before and could be tested as a real moment in the story.

Working method

- Each phase was one branch, built in small steps, with a commit after each step that worked.
- Every step was proved with Vitest and Supertest and, where useful, with one manual request, before the branch was merged into master with --no-ff and pushed.
- Code was written in small pieces, and committed only after testing . The next phase did not start until the current one was proven.
- A development log recorded every piece of work, decision, error and fix with timestamps. It holds the roadmap of each session and is available to reviewers.
- Errors were written down when they were found, including mistakes in the author's own code, and were fixed before moving on.

Roadmap by phase

Phase and date	What was built	Notable decisions and findings
0. Project setup 24 Sep, evening	Repository, master branch, .gitignore before any other file, workspaces (api, web, shared), Zod-validated environment, winston logger with request IDs and redaction, error handling,	Node 22 chosen over Node 20, which reached end of life. The branch was renamed from main to master. Only README.md may be committed as a document.

Phase and date	What was built	Notable decisions and findings
	validation middleware, health endpoint with a database check, Postgres in Docker with a health check.	
1. Dhaka 24 Sep	Drizzle setup, the zones, distances and places tables, and a seed of 12 zones, 66 distances and Tesla stands that is safe to run twice.	Renaming a generated migration also needs the journal tag renamed, or the migrator fails.
2. Jashim 24 Sep	Signup, login and logout with bcrypt and a JWT cookie, role guards, then vehicle registration, going online and offline with an area, and the cast seed for the two drivers.	The vehicle capacity CHECK was added at once. "No vehicle edits during a ride" was left as a known gap until rides existed.
3. Nusrat 24 Sep	The OTP table and identity columns, one-time-code send and verify, simulated NID check at signup, and the passenger seed.	NID and gender were made part of signup. The NID hash is an HMAC, because bcrypt's random salt would defeat the unique index. Twenty parallel guesses were proved to allow only five attempts. OTP endpoints take the phone from the logged-in user, never from the request.
4. Where to and how much 24 Sep	Place search with wildcard escaping, nearest zone and nearest stand with walk time, the service-area check, and the fare service as pure functions with tests of the cast's fares.	The stand's own recorded zone is used for a shared pickup, since the zone nearest the raw point can disagree near a boundary.
5. Request 24 Sep	The rides, ride_requests and ride_events tables with the three unique indexes, an event recorder, and the request service: idempotency with a body hash, one active booking, women-only check, history, cancel and lazy expiry.	Columns for the boarded tick and the price ceiling were added early, though no code used them until later.
6. The pool 25 Sep	The matching service for rules R1 to R6, the accept transaction with the lock, conditional updates and CHECK, private accept, price ceilings, driver cancel and a deadlock retry.	Sixteen tests transcribed every row of the design's own tables and all passed first time. An event write that used the wrong database client inside a transaction was found in the first manual test and fixed.

Phase and date	What was built	Notable decisions and findings
7. The trip 25 Sep	The transition table module, arrive, board, no-show, start with fares locked, per-passenger drop and the stuck-state escapes (cancel 15 minutes after arrival, auto-close after 3 hours).	Twenty-eight tests transcribed every valid and invalid transition. The earlier conditional updates were left in place.
8. Live updates 25 Sep	The connection registry, the notify function, the stream endpoint with heartbeat, the who-is-nudged table, the API Dockerfile and the API service in compose, and the Postman collection.	Events are nudges that carry no data. The heartbeat is a real message. A test-server port collision was fixed by binding to 127.0.0.1.
9. Frontend foundation 25 Sep	Next.js shell written by hand, the /api rewrite, design tokens, the API client, query provider, live-update hook, login and signup screens, demo login buttons, and the web service in compose.	The visual direction was set first. The OTP code had to be shown in demo mode because a Docker-run API could otherwise never verify a new signup. NIDs got their own pepper, separate from the JWT secret, after finding that rotating the JWT secret would have switched off one NID one account.
10. Driver screens 25 to 26 Sep	The vehicle form, the online switch with an area picker, the request list with live updates, and role routing.	The forms reuse the same Zod schemas as the server.
11. Passenger screens 26 Sep	Routing, place search, the nearest stand and walk time, a Leaflet map, the fare quote endpoint, ride options, the request with an idempotency key, and the active booking card.	Two of 42 seeded places sat in a different zone from the nearest zone centre, so a quote endpoint using coordinates was added so the on-screen fare equals the stored one.
12. Ride screens 26 Sep	Driver request cards with fit reasons, accept, boarding, the no-show countdown, start, drop and history. For the passenger: driver, car and co-riders, the stepper, the timeline, the receipt and past rides, with loading, empty and error states.	Logging out does not cancel a booking, on purpose, so a driver still sees it.
13. Seat Race 26 Sep	The demo scenarios behind DEMO_MODE and a demo	Sabotage checks showed the tests detect a fake race, and

Phase and date	What was built	Notable decisions and findings
	key, and the public Seat Race page with both races.	that removing the row lock alone fails nothing.
Refinements after testing 26 to 28 Sep	The signed-in name at the top right, private rides picked up at the stand, Helmet, an online count that leaves out busy drivers, the 15-minute never-arrived rule, the online count shown at pickup, a MapLibre redesign with a full-screen map, and the removal of free map pins.	Each was a rule change from the design, decided after hand testing in four browsers. CORS was deliberately omitted. Unlimited devices per account was accepted as a trade-off.
14. Ship 28 Sep	A fresh clone tested with one command and Postman, the Neon database, the Render service, the Vercel site, and a full trip checked through the live URL.	The first Vercel build failed because the shared package was not declared as a dependency of the web app. It was reproduced locally and fixed.
Pre-release review and fixes 29 Sep	A read-only review of the finished code produced 66 findings and five release blockers. The highest-severity findings and the planned high-severity ones were fixed, each with a failing test first. All branches were pushed.	The next section lists the fixes.
Release 30 Sep	Full suite of 281 tests and web build green. pre-release, release/v1.0.0 and the tag v1.0.0 cut at 125ab61. The uptime monitor started and the full lifecycle checked on the live site.	The README, this document, the video and the slide deck complete the submission.

How the design changed

The design was written seven times. Each version was a discussion followed by a written revision, and each answered problems found by reading the previous one against the brief and against real Dhaka behaviour.

Version	What changed
1	The first complete design. It included a simulated TeslaPay wallet, which version 2 dropped to make time for the safety features.
2	Drivers register their own vehicle. Passengers verify their phone with a simulated one-time code. Passengers choose Share or Private.

Version	What changed
	Women-only rides and a view of who you share with. Drivers choose passengers but cannot remove anyone. A five-minute no-show. Per-passenger drop-off. A rule for cancelling mid-ride. New tables and columns for OTP, gender, ride type and women-only. Rules R5 and R6. A scaling section with numbers. The wallet was dropped to protect time.
3	Renamed Seat Ase?. Women-only was moved from the driver to the passengers: it is worked out from the bookings and is not stored on the ride, so a request can join any all-female ride and no men can join after it.
4	Tesla stands: shared rides are picked up at the nearest real stand, as shared Teslas work in Dhaka today, and private rides at the door. Rule R1 became "same pickup stand". No GPS, with self-declared driver areas. The driver's screen shows a suggested drop-off order and each cash fare. Scenario E was added. Multi-spot pickups and mid-trip refills were ruled out with reasons.
5	Logging moved from pino to winston because it was familiar and easy to defend. An appendix compared Drizzle, TanStack Query, Vitest and Supertest with the tools already known.
6	A red-team review found 45 loopholes in two passes. Private fare became the seat fare times the seats. A price never rises after matching. A boarded tick was added before the start. Empty rides are cancelled automatically. Stuck passengers got escapes. Simulated NID verification with locked gender. Exact door pins are hidden until a driver accepts. Scenarios became scripts. A work sequence was written.
7	This document. It records the released system and every change made once the code existed. The changes are listed below.

Changes made after version 6

Topic	Version 6	Released system	Reason
Node version	Node 20	Node 22, and no dotenv package	Node 20 reached end of life. Node 22 reads env files itself.
NID hashing	A hash of the NID	HMAC keyed by NID_PEPPER	bcrypt's random salt gives a different hash

Topic	Version 6	Released system	Reason
			every time, so a unique index could never fire. A plain hash of a ten-digit number is easy to precompute. A separate key means rotating the JWT secret cannot disable the rule.
Private pickup	Door pickup at the exact pin	The nearest stand, like shared rides	One meeting point for everyone, and no home location is stored.
Private fare	Seat fare times the vehicle's seats	Seat fare times 3, and ceilinged at the quote	A driver with a bigger vehicle would have raised the price after the quote.
Map	Leaflet and OpenStreetMap tiles, search or map pin	MapLibre and OpenFreeMap, full-screen map with a bottom sheet, search only	Restylable, and a tapped point has no name to show.
Driver never arrives	Not covered	Ride cancelled after 15 minutes and passengers return to waiting, with a countdown for the driver	A gap found in testing: an accepted ride with a vanished driver blocked everyone.
Security	Helmet and a CORS lock listed	Helmet added. No CORS package on purpose.	The browser never calls the API cross-origin, so no headers is the strictest setting.
Online count	Online and recently seen drivers	Only cars she could get into, shown as soon as a pickup is chosen	For an app called "is there a seat?" the number must be right.
Fare breakdown	Only the total stored	Breakdown stored at the start and shown on the receipt	The evaluator must be able to check a fare by hand.
Status transitions	A transition table in one module	The same table, now called by every service before it writes	The first build set statuses directly, so the table was not used.
Driver identity	Vehicle only	Drivers verify their phone. Seeded drivers have NIDs and verified phones.	The person collecting women-only passengers should not be the least verified account.
Rides while offline	Blocked in the screen	Blocked in the API too	Rules must not live only

Topic	Version 6	Released system	Reason
			in the screens.
Kick-out	Cancel the whole ride is allowed	A driver cannot re-accept a booking he cancelled	Closes a way to remove one passenger by cancelling and re-accepting the rest.
Demo reset	Reset the demo	Ends only the demo cast's rides and deletes nothing	The first version wiped the whole database, including other users' data.
Distances	Hand-written table looked up on a map	Six exact pairs, the rest estimated and made consistent, in km	Sixty pairs cannot all be looked up in the time available. The estimate is disclosed.
Screens	Separate pages per view	One /ride page and one /driver page	Simpler navigation with the map behind them.
Test database	A compose test profile	A seatase_test database created by the test setup	Simpler, and the demo data is never touched.
Errors	REQUEST_ALREADY_TAKEN and WOMEN_ONLY_MISMATCH	REQUEST_UNAVAILABLE and DOES_NOT_FIT with reasons	One shape for every pooling failure.
Seats	At most 3 per booking	1 to 6 in the database, 1 to 4 on screen	A private booking takes the whole vehicle.
Docker ports	Fixed 5432 and 4000	Every port configurable	docker compose up failed on machines already running Postgres.
Rules as configuration	Environment variables	Named constants in the code	Simpler. The live rule change is a one-line edit.
Sessions	Not specified	Stateless, unlimited devices, no log out everywhere	Correctness comes from database constraints. The token_version fix was scoped and left unbuilt.
Deadline	27 September	30 September	The date moved.

Planned in version 6 and left out

Scenario endpoints for A, C, D and E (they are covered by tests). Completed rides in the seed for history. Database CHECKs that a matched booking has a ride and that pickup and drop-off zones differ, and the "too short for a shared Tesla" message, so a same-zone trip is allowed and charged the base fare. The non-unique lookup indexes. A payment_method column. The phone-verified badge for co-riders. A warning when the nearest stand is over a kilometre away. Total earned in the driver's history. A separate compose file for tests.

Pre-release review

Before cutting the release, the finished code was reviewed against this design in six read-only rounds, without running anything. The rounds found 66 findings, labelled A to BN, and five release blockers. A round found fewer and smaller problems each time, which was the sign to stop reviewing and start fixing. The findings were sorted by severity: 10 critical, 16 high, 24 medium, 7 low, and 9 that only needed a sentence in the documentation. Each fix was written as a failing test first, then fixed, then checked, then merged, in this order.

Finding	Problem and fix
G, malformed ids	A bad id in a URL caused a 500. Ids are validated and return 400.
E, offline during a ride	"No going offline or changing area during a ride" was only a screen rule. The API now refuses it.
F, login limit	The limit was keyed on the raw phone text, so spelling variants bypassed it, and it counted successful logins, so evaluators could lock the shared demo accounts. It now keys on the normalised phone and counts failures only.
A, boarded tick	A driver's cancel left the boarded tick set and a wrong event status. The reset is complete and the cancelled ride keeps no seats.
U, kick-out by cancel	A driver could cancel a ride and re-accept everyone except one person. He can no longer take back a booking he cancelled.
B, private fare	A six-seat vehicle could charge twice the quote. The quote is now the ceiling.
AY, AZ, BB, BH, the data	Gulshan 2 sat 160 m from Banani, the distance table broke the shortest-path rule 15 times, and a seed fix could not reach an existing database. The centre was corrected, distances made consistent, the seed made to update, and tests added for the data.
H, demo reset	The reset wiped the whole database. It now touches only the cast.
AL, transition table	The table was dead code. Every service now calls it and the table was corrected to the real actors.
I, AM	One-time codes used Math.random, now crypto.randomInt. A long idempotency key returned 500, now 400 above 100 characters.
X, M, driver identity	Drivers never verified their phone, and the seeded drivers skipped the signup rules. Both were fixed.
BE, cold start	The first visit after a quiet spell showed an error. Requests retry, a Waking up card

Finding	Problem and fix
	explains, and a monitor keeps the server awake.
J, BK, deadlines	The driver could not see the 15-minute arrival clock, and countdowns used the phone's clock. The server now returns the deadlines.
L, V, expiry	A waiting screen never expired, expiry could block rebooking, and expiry left no history. All three were fixed.
BM, request cards	A private request looked like one seat and no card showed a fare. Cards now show "whole Tesla" and the fares.
BL, receipt	The receipt showed only a total. The breakdown is stored and shown.
K, online count	The count included cars she could not get into. It now counts only usable cars.
BI, AB, ports	docker compose up failed when Postgres already ran on 5432. All ports are configurable and the tests follow DB_PORT.
RB2 and RB3	All local branches were pushed, and pre-release, release/v1.0.0 and the tag were cut.

One of the fixes caused a new bug that testing on the live site caught the same evening. Retrying the "who am I" request for up to a minute during a cold start left the page empty while it waited, and the Waking up card was added to fix that. The review also corrected itself once. An early finding advised removing the published database port from compose, and a later round noted that the tests run on the host and need that port, so the port was made configurable instead.

Known limitations

These were reviewed and left unfixed before release. Each is either documented or listed under next improvements.

- NID and gender verification are simulated. Any valid-format number is accepted, and in demo mode the one-time code is shown on screen. The women-only rule therefore protects against accidents and does not stop someone who lies. Production would check a real ID registry and send real SMS.
- Pickup is always the straight-line nearest stand. If a Tesla is filling at a slightly farther stand, the passenger is not shown that option. Several zones have a single stand, so the situation cannot arise there in the demo.
- One role per account, since one NID allows one account.
- There is no in-app way to contact the other person, and no passenger "I'm here" signal, so a no-show rests on the driver's word.
- A driver cannot undo a boarded tick made by mistake before the start. A wrongly boarded passenger waits for the drop-off or the three-hour close.
- Some driver and booking responses return the full booking row, including internal fields such as the idempotency key, and a passenger who ends badly (expired, no-show) is

shown only one word in history, with no explanation card.

- The same time rules (15 minutes, 5 minutes) are still defined in more than one file.
- Registering a phone number, NID or plate reserves it before verification finishes. Someone could tie up a number they do not control.
- Signup and login are limited per phone number with no global limit, and a price ceiling can be used by a second account that joins and cancels.
- Cleanup jobs run lazily on read, and a few lookup tables are queried directly. That is fine for a demo and is discussed in the scaling section.
- Sessions cannot be ended everywhere, and a stolen token works until it expires.
- Stand coordinates are small offsets from zone centres and were not surveyed, and the distance table is an estimate. The live connection list works for a single server.
- Free hosting means the API can sleep. The monitor and the Waking up card reduce the effect.

Next improvements

Driver location tracking and routing. Choosing among nearby stands, with the one that has a Tesla and free seats shown first. A token version to end all sessions. Real SMS and a real ID registry. An "I'm here" and "running late" signal, and an undo for the boarded tick. Centralised rules that the API returns to the screens. Lookup indexes and scheduled cleanup jobs. A frontend test suite. Female drivers, an SOS button and a share-trip link. Multi-server live updates through LISTEN and NOTIFY.

Process and use of AI tools

Claude was used to brainstorm the system design through its seven versions, and later to review the finished code against the design, which produced the 66 findings. Every piece of code and every decision was checked manually and verified by writing test cases, and a change was committed and pushed only after its tests passed. Most commit messages were written with the commit-message shortcut in VS Code, which reads the staged changes. The full roadmap and log of every session, including what was built, what broke, what was fixed and what was invented, is kept in the development log (devlog.md), which reviewers can read.

Two decisions show how suggestions were treated. For hashing the NID, the first idea was bcrypt because passwords already use it, and it was rejected: bcrypt salts randomly every time, so the same NID hashes differently and a unique index can never catch a duplicate, while a plain hash of a ten-digit number is easy to precompute. The result is an HMAC keyed by a server secret. For the rule that stops a driver removing someone by cancelling a ride and re-accepting the rest, the review suggested blocking re-acceptance for 15 minutes. The build instead checks whether the driver already cancelled that specific booking, with no timer at all. That needs no new setting to explain, and if the same passenger books again later it is a new booking that any driver may take.

Submission deliverables

Deliverable	Status
Working MVP: frontend, backend, database	Released as v1.0.0 and live at https://seat-ase-web.vercel.app .

Deliverable	Status
Docker, .env.example, no secrets committed	docker compose up -d --build starts everything. .env.example holds placeholders only.
Migrations and seed with the story cast	Nine migrations, and a seed with the five cast members, two Teslas, 12 zones and 42 places.
Architecture diagram and ERD	In the README as Mermaid diagrams, and in this document as tables.
Git branches and history	master, pre-release, release/v1.0.0 and the feature and fix branches, merged with --no-ff.
Tests	281 tests in 27 files, and a Postman collection.
README with AI usage	Written, with screenshots. Links to this document and the development log are added at submission.
Six-minute video	Scripted in three segments: the problem, the engineering and a product tour. A short slide deck supports it.
Scaling bonus	Included in the scaling section of this document and in the README.

Appendix A: Design questions and short answers

Question	Short answer
Why Postgres and not MongoDB?	Relationships and seat safety. The database enforces capacity, uniqueness and valid links, so bugs and races cannot break them.
Walk me through the last-seat problem.	Lock the ride row, re-check fresh numbers, apply a conditional update, and let the CHECK constraint be the final guard. The loser gets a 409 and nothing changes for them.
Why not check seats in JavaScript before saving?	Two requests can both check before either saves. The check and the save must be one atomic step in the database.
Why two status chains?	A pool has one car trip and several bookings. One passenger cancelling should not change the ride's status.
Why lock the fare at the start?	Before the start the group can change. After it the group cannot, and a price should not change mid-ride.
How does Rafiq not see Nusrat's fare?	Every query is scoped to the logged-in user, and the live nudge carries no data, only a reason to re-fetch through the API.
What if the API crashes mid-accept?	The transaction never committed, so Postgres rolls everything back and no seat is half-booked.
Why can't the driver remove someone?	Otherwise a driver could remove a person for their looks or gender after they stopped looking for another ride. The no-show after five minutes

Question	Short answer
	is the one exception.
How is women-only enforced, and what is its weakness?	Only passengers turn it on. Rule R5 is checked in the accept transaction after the ride is locked, using the bookings already in the ride. Gender is self-declared, and the production fix is a real ID registry.
Why can't a freed seat be filled mid-trip?	Without the Tesla's location we cannot tell if a new pickup is on the way, and per-segment fares could not be checked by hand.
How does it scale to 1M users?	Bookings are about 25 a second. Location pings and open connections are what break, so those are scaled in stages, when metrics say so.
Change the pool radius from 3 km to 2 km, live.	It is one constant, MAX_DROP_KM. The test for Mohakhali and Gulshan 1 at 2.5 km now fails, which shows the rule doing its work.
What would you change first for production?	Driver location tracking, real identity and SMS, end-of-session control, scheduled cleanup and centralised rules.

Appendix B: Demo accounts and fixed numbers

Name	Role	Phone	Notes
Nusrat	Passenger	01700000030	Banani to Mohakhali
Rafiq	Passenger	01700000031	Banani to Gulshan 1
Shirin	Passenger	01700000032	Banani to Mohakhali, women-only
Jashim	Driver	01700000010	Bullet, 3 seats
Mokbul	Driver	01700000011	Toofan, 3 seats

The password for every account is password123. Fixed fares in poysha: Nusrat pooled 9625, Rafiq pooled 7750, Nusrat alone 11500, Nusrat private 34500, and in scenario E Rafiq 7750, Nusrat 9625 and Shirin 11500. Two evaluators should not use the same demo account at the same time, since one active booking per passenger and one active ride per car would make them collide. A fresh signup works alongside the cast. The demo key for the Seat Race is shared privately and is not written here.